Delegated Coding

Managing your AI collaborators

Anna Woodard, PhD

UChicago Data Science Institute · UChicago Medicine

2026-06-18

scan to download these slides

Is generative AI good or bad?

Real costs — energy and water, job disruption, concentration of power, consent for training data.

Real benefits — productivity/speed across knowledge work, wider access, etc.

These are big questions, and society is still working them out. **That's not what today is about** — but I'm not pretending they're settled.

Today's narrower question: *if* you choose to use these tools, how do you use them **responsibly, and well?**

Why not call it “vibe coding”?

Vibe coding: prompt, accept, run, hope.

Fine for a throwaway script — not fine when someone has to **trust the result**.

Delegated coding: the AI does the work; **you stay in charge and accountable**.

The skill you’ll need this summer is not prompting. It’s **management**: scoping tasks, setting standards, reviewing the work.

First, the rule that outranks this talk

⚠️ **Talk to your mentor — in week 1, not week 7.** Three questions to ask:

1. *"Are you OK with me using AI tools — and which ones, for what?"* (code? analysis? writing?)
2. *"What are the rules for our **data** — and any **unpublished code** — what can the tools see, and how should I **disclose** AI use?"*
3. *"By the end of the summer, **which parts of this project should I be able to explain** entirely on my own?"*

Nothing in the next hour overrides what your mentor tells you.

What this hour is — and isn't

This is **not a how-to workshop**. No tool walkthrough, no syntax, no setup guide.

And *which* tool? Don't agonize — they're all good, the "best" shifts monthly and differs by person and project. Try a few; use whatever your school gives you cheapest access to.

Why: for any *"how do I…?"* question, your best teacher is **the agent itself** —

- it knows the current version of the tool (these change *monthly*; slides don't)
- it answers *your* question, on *your* project, at the moment you're stuck

What an hour *can* give you is the part the agent won't: **judgment** — what to delegate, what to demand back, what to check, what to own.

Part 1: What you're actually working with

Under the hood: a language model

A large language model (LLM) is trained in two stages — neither objective is *"be correct"*:

1. **Predict what comes next**, over a huge amount of text and code — where it learns to write code, explain errors, follow instructions.
2. **Satisfy human raters.** People compare candidate answers; the model is tuned toward the ones people prefer.

Each stage leaves a fingerprint on the output:

- stage 1 rewards the **plausible** — which usually, *not always*, matches the **true**
- stage 2 rewards what **readers approve of** — a pull toward **confident, fluent, agreeable**

Newer models are better calibrated — they push back and say *"I'm not sure"* more than they used to. But: **fluent and confident ≠ correct** — least of all on *your* data and code, which it can't verify alone.

You’ve probably met three generations of this

	Tool	How it works	You delegate…
1	**autocomplete** (Copilot)	suggests as you type	a line
2	**chat** (ChatGPT)	you paste code in, copy answers out	a function
3	**agents** (Claude Code, Cursor, Codex)	works *in your project* on its own	a **task**

Generations 1–2: you are the hands, every step passes through you.

Generation 3: the AI is the hands. **That changes your job.** This talk is about generation 3.

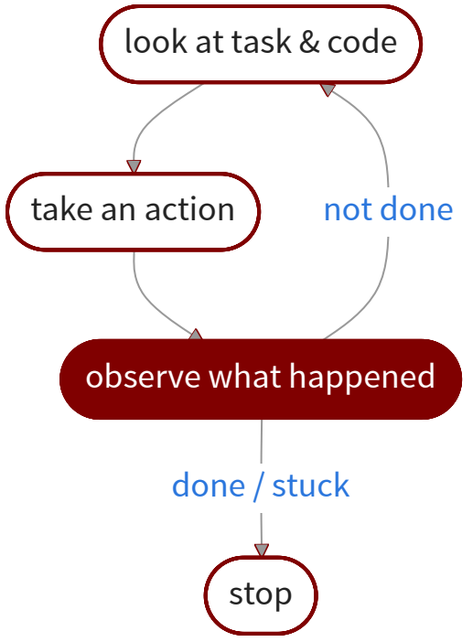
What turns a chatbot into an agent

An agent = a language model + **tools** + a **loop**.

Tools — actions it can take in your project:

- read and edit files
- run shell commands
- search your codebase and the web

The loop — the engine:



The loop is the big deal: it runs your code, **reads its own error messages, and fixes them** — for minutes to hours, without you watching.

One more concept: the context window

Everything the agent currently knows — your instructions, the files it has read, the commands it has run — sits in its **context window**: its working memory.

Three consequences you'll feel immediately:

- it's **finite** — when it fills, the tool **auto-compacts** (summarizes to keep going), but that's **lossy and silent**: details quietly drop
- each session starts **fresh** — and any "memory" it keeps is partial and **not yours to audit**
- it only knows what it has **read** — it won't know about your data's quirks unless told

So anything that has to be **right every time**, you write down in the project — where you **control it, version it, and can check it.** That's not a workaround — it's *the* system. The single most important habit in this talk falls straight out of it: in Part 4, the agent keeps a **lab notebook**.

What it's genuinely good and bad at

Strong — often better than you'd expect

- changes across **many files**, not just snippets
- a bug **end to end**: reproduce → fix
- learning an **unfamiliar codebase**, explaining it back
- messy notebook → clean, **reproducible** script
- tedious-but-defined: **parsers, tests, wrangling**
- **making code run faster** (profiling, vectorizing)
- wiring up **cluster jobs** (SLURM)

Weak — exactly where research lives

- your **intent** — what you didn't say
- your **data's quirks**
- what it **doesn't know** about *your* problem
- whether a result **makes sense**
- genuinely **novel** methods, no precedent to copy

The strong column is most of your coding hours this summer — **and it keeps growing.** The weak column was always your job — and **stays** yours: it's about *your* context, not the model's horsepower.

Part 2: Research is a different game

You've been trained on coursework. Research is different.

Class assignment

- the answer is **known**
- autograder / TA checks you
- the **code** is the product
- wrong code → fails visibly → you fix it
- ends when it passes

Research

- **nobody knows** the answer
- *you* are the checker
- the **claim** is the product — code is just the instrument
- wrong code can **run perfectly** and produce a wrong claim
- ends when you can **defend** the result

Part 3: You are the PI

The mental model

Treat the agent as a fast, tireless, overconfident **junior collaborator** — and yourself as the **PI** (the principal investigator: the person who runs a lab).

A PI doesn't watch every line get typed. A PI owns:

- the **question** being asked
- what **evidence** would answer it
- what would **change our minds**
- the **constraints**: which data may be used, what counts as success

Sound familiar? It's how your **mentor** will manage *you* this summer — watch what good delegation feels like from below, then apply it from above.

One difference from a real student: the agent **won't reliably carry the lessons forward** unless *you* write them down.

The prompt shift

Two ways to lose

- *Micromanaging* — “fix this line… no, the other axis” → you’re **the hands, just slower**
- *Vibe-delegating* — “just build the whole thing” → you **can’t vouch** for what comes back

Delegating — in charge, not in the weeds

> “Here is the **goal**, and **what would convince me** it worked. **Inspect** first, propose a **plan**, implement the **smallest thing**, run a **check I can rerun**, and tell me **what changed**.”

The win isn’t *hands-off* — it’s **you didn’t type it, but you understand it and can defend it.**

What would convince you?

Before you delegate, decide what would make you **believe it worked** — concretely enough to check.

Vague — you can't tell if it's right

- "make a plot of the results"
- "clean up the data"
- "speed up this script"
- "build a good model for X"

Acceptance criteria — you can

- "plots **x vs y**, one series per group, axes labeled"
- "row count = 412 in, 412 out — or tell me why not"
- "**same outputs**, just faster"
- "**beats baseline +0.02 AUC** — survives a new seed"

Your turn: pick a task from *your* project — analysis, simulation, imaging, scraping — what would convince *you* it worked?

Can't say what would convince you? You're **not ready to delegate** — working that out *is* the research.

Agree on the plan before any code

For anything non-trivial: **“Plan first — don’t write code yet.”** The agent drafts a short design (a **design doc** for bigger tasks); you catch the misunderstanding while it’s still **one paragraph, not 300 lines.**

> “Before implementing, tell me your **approach**, the **files** you’ll touch, the **assumptions** you’re making, and **how you’ll check it**. Wait for my OK.”

You’re checking three things:

- it understood the **real** task — not a plausible nearby one
- the approach is one **you’d endorse** — no odd shortcuts, no scope creep
- its **assumptions** match your data and your constraints

Five minutes on the plan beats an hour reviewing code that solved the **wrong problem.**

What to delegate, what to own

Delegate — implementation & mechanical investigation

- “make this plot from this CSV”
- “write a parser for this format”
- “turn this notebook into a script”
- “find why this job crashed”
- “summarize what this code does”
- “check these folders for differences”

Own — question, design, interpretation

- “is this result *real*?”
- “is this comparison *fair*?”
- “what does this *mean*?”
- “what should we try *next*?”

For these, use the agent as a **skeptical assistant** — “argue against my result” — never as the decision-maker.

Rough rule: if the task has a checkable “done”, delegate it. If the task **is** the judgment, it’s yours.

Part 4: Running the lab

Eight habits for delegating work you can still vouch for — none of them are about typing code.

1. Put the project in git

Git = version control: **snapshots of your project** you can compare and restore. If you've never used it: one-day learning curve, ask the agent itself to teach you.

For agent work it's non-negotiable — the agent **edits your files and runs commands**:

- **commit before every delegated task** — cheap checkpoints
- broke something? **revert.** git is your **undo button**
- every result traces to the **exact commit** that made it — your **audit trail**

The whole point is the safety net: **revert anything, trace everything** — so you can delegate boldly.

2. Write the onboarding doc: `AGENTS.md`

A plain text file in your project that the harness **reads automatically at the start of every session**. (Claude Code also reads `CLAUDE.md`; Cursor has rules files — same idea.)

Remember: the agent wakes up **fresh** every session. This file is what survives.

Put in it what you'd tell a new lab member on day one:

- how to set up and run things (environment, commands)
- **where the data lives** — and what must never be touched or uploaded
- what counts as a **valid result**, where outputs go, naming conventions

Start with ten lines. How it grows is the interesting part →

Where the rules come from: something goes wrong *once*

The habit: every time something goes wrong, don't just fix it — go to `AGENTS.md` and **encode the general version**, so the whole *category* can't happen again.

✗ It analyzed `data_old.csv` — two similar files were sitting in the folder.

> "Print the full path of every input actually loaded. If several candidates exist, **stop and ask**."

✗ The patient count quietly went 412 → 389 between runs (different missing-data handling).

> "Report the **N after every filtering step**. Any change in counts between runs must be **called out explicitly**."

Inside a real one: excerpts from my `AGENTS.md`

> “Fail fast. No silent fallbacks, no hidden defaults.” — errors should surface, not quietly corrupt results

> “Do not call something an A/B test unless **only one knob changed**.” — guards every comparison I’ll ever be shown

> “Don’t run experiments that **won’t change the decision.**” — design for the result that changes your next move

When a lesson outgrows the project: skills

`AGENTS.md` is memory for *one* project. Lessons that generalize become **skills**: small packaged workflows — Claude Code, Cursor, Codex, Gemini CLI.

Each skill advertises just a one-line **title** — a few tokens the agent always sees — and loads its **full instructions only when a task needs them**. So you don't say *"use this"*: the titles tell it what's available, and it reads the right one itself. **Same vetted workflow every conversation — and no tokens spent loading what you're not using.**

I've published mine, free to install — **uchicago-dsi/ai-sci-skills**:

- `$sensemaking` — does a finding **hold up before I trust it?**
- `$skeptical-labmate` — **stress-test** a claim or baseline
- `$lab-notebook` / `$handoff` — records that stay **restartable**

3. Demand a report, not just code

Don’t accept just code — **demand a report at the end**:

1. the **commands** that were run
2. the **artifacts** produced (files, figures, tables)
3. **what changed**
4. the **remaining uncertainty** — what it assumed, what it couldn’t check

“Done, everything works” is a **worse** answer than “done, but I assumed the dates are UTC — you should check that.”

4. Give it a target it can check itself

You say it's wrong. It tries again. Still wrong. You correct it again — **you've become the test**, grading every attempt by hand, one round at a time.

The fix isn't a sharper correction. It's a **check the agent can run without you** — then: *"keep iterating until it passes."*

Free — it already loops on these:

- a crash → the **stack trace**
- a failing **test** or **type error**
- a **linter** complaint

You have to build these:

- "rows must still sum to N" → an **assertion**
- "match this hand-checked case" → a **diff**
- "accuracy > 0.9 on held-out data" → a **metric**

Now **the agent** iterates, not you — you set the bar **once.** *Not a new gadget: it's the Part 1 agent-loop pointed at a finish line you drew — and only where "right" is checkable.*

5. Make every result auditable

One **result** → **one folder** → a `README` of how it was made. A "result" is whatever your project produces — a model run, a simulation, a dataset, a figure, a deployed tool.

The rule from my own `AGENTS.md`, sized for a summer project — paste it into yours:

> "Every new results directory must contain a `README.md` **before the result is reported**. At the top: one line on **what this is for**. Then enough to rerun or audit it: the exact **command**, any **config or parameters**, the **input files**, the **git commit**, and **how to regenerate it**. Add key plots and numbers as they appear."

Why: week-7 you, bug found, mentor asking "which version made this figure?" — this turns a crisis into a 10-minute rerun.

6. Make it keep the lab notebook

Two records, easy to confuse — you want **both**:

- a result's **README** = how *one* result was made *(provenance)*
- the **lab notebook** = the *thread across all of them (the story)*

One rule in `AGENTS.md`, and the agent keeps the notebook as it works:

> "After each experiment or work session, append to `LAB_NOTEBOOK.md`: what you tried, the result, and what it changes about **what we believe** and **try next**."

The agent starts each session **fresh** — so the notebook is its memory. But it's **yours** too: you stop **going in circles** (you can see what you already tried), you walk your **mentor** through *what you did and what you found*, and by August it's your **poster**, written as you went.

Inside a real one: a `LAB_NOTEBOOK.md` entry

One dated, append-only entry — what the agent appends after a run:

```
## 2026-07-14
Tried:    logistic baseline, cleaned cohort (n=412) → AUC 0.71
          provenance: results/2026-07-14_logit/
Surprise: 23 rows dropped for missing subject_id —
          ask mentor: excludable, or a merge bug?
Believe:  signal real but weak — features are the bottleneck, not the model
Next:     add the lab values Dr. K mentioned; re-run before the GNN
```

Every entry, the same four moves: **what I tried · what surprised me · what I now believe · what's next.**

7. Maintain the workflow itself

Weeks of agent work accumulate **slop**: duplicate scripts, dead code, stale instructions, three versions of the same plot function.

Every week or two, delegate a cleanup pass:

> "Read `AGENTS.md` and the recent lab-notebook entries. Find instructions that are outdated, duplicated, or missing given recent failures. Propose a shorter, cleaner version before editing anything."

Ground rule for cleanup passes: **results must not change** — only make the next experiment easier to run and audit.

8. Let it run the campaign — overnight

For projects with many computational runs — model training, simulations, sweeps:

You hand off the **plan**; the agent runs the **loop**:

- you set the **question**, the **grid** (what varies), a **success metric**, a **budget**
- it writes the **SLURM** script, launches, and **babysits the queue** — reads failed logs, fixes, resubmits
- each finished run lands in its own **README'd folder** + a line in the **lab notebook**
- it stops when the **metric is met** or the grid is exhausted

You wake up to a table of **reproducible** runs and a notebook saying **what won, and why.**

⚠️ Autonomy multiplies **mistakes** too — a wrong metric or a silent bug now runs 200 times. **Pilot one run, audit it, *then* scale.** *(Which is exactly Part 5 →)*

Part 5: When it goes wrong

It will be wrong in ways that *look* right

Remember: fluent and confident **by construction**. The dangerous failures don't crash — they run clean and lie quietly:

- two runs **share a hidden assumption**, so their agreement is fake evidence
- the summary sounds **more confident than the evidence supports**

Don't go hunting in the code — these surface only in the **outputs**: the **row counts**, the **file actually loaded**, the **config**, the **summary's wording**.

Five minutes per task. That's the entire price of staying in charge.

Case study: the seed that didn't vary

I asked my agent to rerun an experiment **three times**, to check the result was stable and not a fluke.

It reran it three times — with the **same random seed**.

(The seed fixes the "random" choices — data shuffling, initialization — so the same seed means the same "random" numbers every time.)

Three matching results. **Zero independent evidence.** And it *looked* like exactly what I asked for.

The fix wasn't "be more careful." It was a new rule written into `AGENTS.md`:

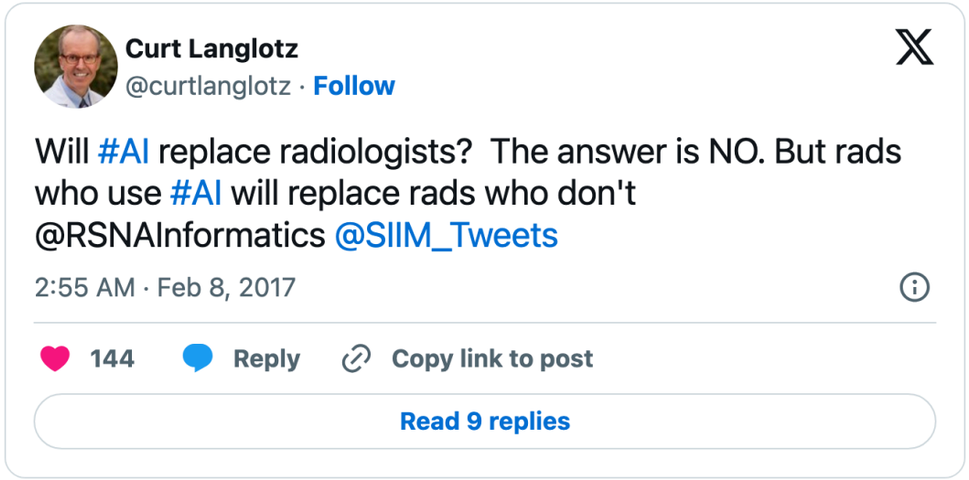
> "Before running comparisons or repeats, state **what is supposed to vary** and **what must stay fixed** — and afterwards, **check the run records** to confirm it."

You can't list the failure modes in advance — **harvest them.**

Part 6: The student catch

“Is this thing going to take my job?”

The oldest version of the honest answer — from radiology, 2017:



> “AI won’t take your job. It’s somebody using AI that will take your job.” — Richard Baldwin, economist, WEF 2023

Become the somebody!

Don’t delegate the learning

A PI can only sign off on what they can **evaluate**. The catch: in a new field, a right answer and a **confident wrong one look identical** — so you don’t know what you can’t check.

“Run a t-test” → **p = 0.03**, looks clean. Miss that it’s the **wrong test for skewed data** and you’d never know. So how do you find the gap?

Make the agent show its reasoning, not just its answer:

> *“What did you decide here, what are you assuming, and how could it be wrong? Explain it so I could defend it to my mentor.”*

Now *“I assumed the groups are roughly normal”* is **on the table** — a gap you can learn, or take to your mentor. **That’s how the boundary moves.**

Make it a tutor, not a vending machine

You have unlimited access to a patient expert with no ego and no office hours. Most students only ever ask it for *output*. Ask it to *teach*:

- *“Why this approach over the alternatives?”*
- *“What could go wrong with this?”*
- *“Walk me through this diff like I’m the reviewer.”*
- *“Quiz me on this code until I can explain it.”*

For a student, the explanation is worth more than the code.

Verify on something you can check

Before trusting it on the case you *can't* check, run it on a small case where you **already know the answer**.

- analysis script → run it on 10 rows you can verify by hand
- statistical result → test it on fake data **built to have a known answer**
- parser → feed it one file you've read yourself

Right on the known case → it has *earned* some trust on the unknown one.

Ground rules

- **Your data doesn’t have to leave.** Your prompts and code go to a company’s servers; your **data** doesn’t have to. Never point a cloud tool at patient, regulated, or DUA-covered data. When in doubt, ask your mentor first.
- **Clear the project’s code with your mentor first.** Most research code is unpublished — that’s normal; just get their OK before sending it to a cloud tool. You can still use the agent *around* it: concepts, debugging from the **error message**, pseudocode — a tutor and planner, no source handed over.
- **You own what you submit.** Every line is yours, whoever typed it. “The AI did it” is never an excuse.

Part 7: Getting started

Your first delegated task — this week

Pick a **small, annoying, real** task — about what you'd bring to **one mentor meeting**:

1. Put the project in git; write a 10-line `AGENTS.md`
2. Delegate with the template: goal, context, **what would convince you**
3. Require the report: commands · artifacts · what changed · **uncertainty**
4. Audit the outputs (not the code): right data in? key number sane? row count unchanged — or the change explained? — five minutes
5. First mistake you catch → first rule in your `AGENTS.md`

Don't start with the big open-ended project. **Start where you already know what "done" looks like.**

If you remember three things

1. **You are the PI.** Delegate implementation; own the question, the design, and the interpretation.
2. **Keep the lab notebook.** You can't rely on the agent's memory — so the project's learning has to live in the repo. The notebook is where it lives (`AGENTS.md` and provenance feed it); every caught mistake becomes a rule.
3. **Trust is earned by verification.** Don't just check that it *ran* — check what it *produced*: the right data went in, the headline number passes a smell test, the dataset didn't change size without a reason. Then verify on a case where you already know the answer.

Questions?

Guides — don't hunt for the perfect tutorial; skim one of these, then learn on a real task: Claude Code · Codex · Cursor